Python vs NumPy

- In Python, each operation inside a loop is executed separately. In every iteration, the interpreter re-evaluates the instruction and checks the data types it is working with, which naturally increases execution time.

- In NumPy, the loop is not executed inside Python itself. You only call the function from Python, while the actual computations are performed inside a library written in C. The operations are carried out there much more efficiently. This approach is known as vectorization.

- In NumPy, data is stored in contiguous memory arrays, which makes access faster and more efficient.
  In contrast, in standard Python lists, elements are stored as separate objects in different memory locations, and the list only keeps references (pointers) to those objects.

  - **Each element in Python is a full object that contains additional metadata.**
  - **In NumPy, the elements are stored as raw numeric values.**
  - **This reduces memory consumption and improves CPU performance.**

- NumPy takes advantage of system and processor optimizations such as:

  - ➢ **SIMD (Single Instruction, Multiple Data):** a technique that allows a single instruction to operate on multiple data elements simultaneously.

  - ➢ **BLAS (Basic Linear Algebra Subprograms):** an optimized library used for performing mathematical operations such as matrix multiplication, dot products, and other linear algebra operations efficiently.

  - ➢ **Multithreading:** where the workload is distributed across multiple CPU cores to speed up execution.

Therefore, NumPy is much faster and more efficient for large datasets because it executes operations in optimized compiled code and makes better use of memory and CPU resources compared to pure Python loops.

Sources:

- https://numpy.org/doc/
- ChatGPT
- https://docs.python.org/3/